

Adigraph, v1.7.1

Luca Cappelletti

December 2018

Contents

1	Introduction	3
1.1	What is Adigraph?	3
1.2	License	3
2	Setup	4
2.1	Installing the dependencies	4
2.2	Installing Adigraph	4
3	Usage	5
3.1	Creating a new graph	5
3.2	Changing an existing graph	5
3.3	Adding nodes	6
3.3.1	Custom node colors	6
3.3.2	Custom node width	6
3.3.3	Custom node labels	6
3.4	Automatically position nodes	7
3.4.1	Colored automatically positioned nodes	7
3.5	Adding edges	8
3.5.1	A simple edge	8
3.5.2	A looped edge	8
3.5.3	A colored simple edge	8
3.5.4	A wider simple edge	9
3.5.5	A weighted edge	9
3.5.6	A weighted edge with label	9
3.5.7	Edge in both directions	10
3.5.8	Edge with weights in both directions	10
3.5.9	Positioning labels	10
3.5.10	Positioning weights	11
3.5.11	Multiple edges with weights	11
3.6	Kleene star operators	11
3.6.1	Kleene star on an element	11
3.6.2	Kleene star minus the element	12
3.6.3	Combining Kleene operations	12
3.7	Paths	13

3.7.1	Augmenting paths	13
3.7.2	Custom colored augmenting Paths	16
3.7.3	Custom colored Paths	17
3.8	Cuts	17
3.8.1	Colored cuts	18
3.9	Non oriented (undirected) edges and custom edge stiles	18
3.9.1	Non oriented (undirected)	19
3.9.2	Dashed	20
4	PyAdigraph	21
4.1	Installation	21
4.2	Example	22
4.2.1	Python code	22
4.2.2	Latex result	23
5	Utilities	25
6	Warnings	26
6.1	Reserved words	26

Chapter 1

Introduction

1.1 What is Adigraph?

Adigraph is a latex library for drawing directed graphs and augmenting directed graphs, and to draw cuts over them.

It handles automatically the positioning of labels, with the exception of the horizontal position, and the inclinations of cuts.

The latest version is available on [Github](#).

1.2 License

Copyright 2018 Luca Cappelletti

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sub-license, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Chapter 2

Setup

2.1 Installing the dependencies

Clearly you need to have texlive installed. Then, make sure you have the following packages:

fp Used for floating point calculations.

xparse Used for elaborating parameters.

xstring Used for elaborating strings.

etoolbox Used for operations on lists.

tikz Used for drawing the actual graphs.

tikz calc library Used for some internal calculations in tikz.

To be sure you can run the following, that will install the packages only if they are not already present:

```
1 | sudo tlmgr install etoolbox fp xstring
```

2.2 Installing Adigraph

You can install Adigraph, if it isn't already present in your setup, by running the following on Unix systems:

```
1 | sudo tlmgr install adigraph
```

On windows you should check on your package manager of choice (some latex distribution have a tlmgr implementation on windows too.)

Chapter 3

Usage

3.1 Creating a new graph

Here we create a new Adigraph object, called *myAdigraph*.

```
1 \NewAdigraph{myAdigraph}{  
2   <nodes here, separated by semicolon>  
3 }{  
4   <edges here, separated by semicolon>  
5 }{  
6   <cuts here, separated by semicolon>  
7 }[  
8   <edge style here>  
9 ]
```

3.2 Changing an existing graph

You can renovate an older graph by calling `\RenewAdigraph`

```
1 \RenewAdigraph{myAdigraph}{  
2   <nodes here, separated by semicolon>  
3 }{  
4   <edges here, separated by semicolon>  
5 }{  
6   <cuts here, separated by semicolon>  
7 }[  
8   <edge style here>  
9 ]
```

3.3 Adding nodes

We set its nodes with the following syntax: *node name*[, *textual color*, *border width*]: *x coordinate*[, *y coordinate*][: *label*]_{*g*}.

```
1 \NewAdigraph{myAdigraph}{  
2     s:0,0;  
3     t:4,0;  
4 }  
5 \myAdigraph{}
```



3.3.1 Custom node colors

To color a node you can use the following syntax: *node name*[, *textual color*]: *x coordinate*[, *y coordinate*]_{*g*}. For example, to draw s in red and t in blue we would write:

```
1 \NewAdigraph{myAdigraph}{  
2     s,red:0,0;  
3     t,blue:4,0;  
4 }  
5 \myAdigraph{}
```



Tested available colors are: red, blue, black, green. You may extend the possible colors with LaTeX libraries such as xcolor.

3.3.2 Custom node width

To color a node you can use the following syntax: *node name*[, *textual color*[, *border width*]]: *x coordinate*[, *y coordinate*]_{*g*}. For example:

```
1 \NewAdigraph{myAdigraph}{  
2     s,red,5:0,0;  
3     t,blue,3:4,0;  
4 }  
5 \myAdigraph{}
```



3.3.3 Custom node labels

To add a custom label you can use the following syntax: either *node name*: *x coordinate*[, *y coordinate*][: *node label*]_{*g*} or *node name*[, *textual color*]: *x coordinate*[, *y coordinate*][: *node label*]_{*g*} will work:

```

1 \NewAdigraph{myAdigraph}{
2   s,red:0,0:start;
3   t:4,0:end;
4 }
5 \myAdigraph{}

```



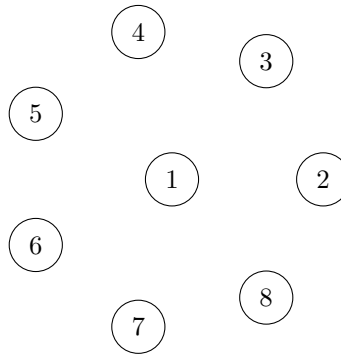
3.4 Automatically position nodes

When no coordinates are given or you just don't have time to think about where to put those nodes, just choose a radius and Adigraph will position them on the circle of that radius.

```

1 \NewAdigraph{myAdigraph}{
2   1:0,0;
3   2:2;
4   3:2;
5   4:2;
6   5:2;
7   6:2;
8   7:2;
9   8:2;
10 }
11 \myAdigraph{}

```

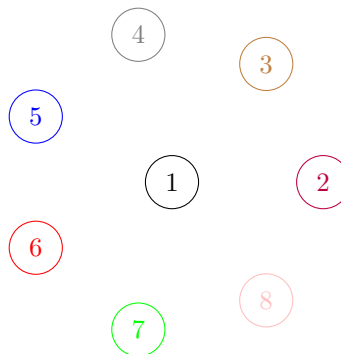


3.4.1 Colored automatically positioned nodes

```

1 \NewAdigraph{myAdigraph}{
2   1:0,0;
3   2,purple:2;
4   3,brown:2;
5   4,gray:2;
6   5,blue:2;
7   6,red:2;
8   7,green:2;
9   8,pink:2;
10 }
11 \myAdigraph{}

```

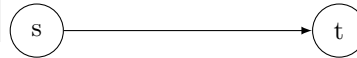


3.5 Adding edges

We set its edges with the following syntax: *ifirst node, second node,[color,[edge width]][[:weight[:label[:label position]]]]*.

3.5.1 A simple edge

```
1 \NewAdigraph{myAdigraph}{  
2     s:0,0;  
3     t:4,0;  
4 }{  
5     s,t;  
6 }  
7 \myAdigraph{}
```



3.5.2 A looped edge

Looped edges position automatically by themselves to minimize overlapping.

```
1 \NewAdigraph{myAdigraph}{  
2     s:0,0;  
3     t:4,0;  
4 }{  
5     s,s;  
6     t,t;  
7     s,t;  
8 }  
9 \myAdigraph{}
```



3.5.3 A colored simple edge

```
1 \NewAdigraph{myAdigraph}{  
2     s:0,0;  
3     t:4,0;  
4 }{  
5     s,t,red;  
6 }  
7 \myAdigraph{}
```



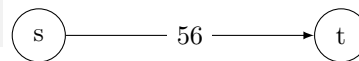
3.5.4 A wider simple edge

```
1 \NewAdigraph{myAdigraph}{  
2     s:0,0;  
3     t:4,0;  
4 }{  
5     s,t,red,5;  
6 }  
7 \myAdigraph{}
```



3.5.5 A weighted edge

```
1 \NewAdigraph{myAdigraph}{  
2     s:0,0;  
3     t:4,0;  
4 }{  
5     s,t:56;  
6 }  
7 \myAdigraph{}
```



3.5.6 A weighted edge with label

```
1 \NewAdigraph{myAdigraph}{  
2     s:0,0;  
3     t:4,0;  
4 }{  
5     s,t:56:myLabel;  
6 }  
7 \myAdigraph{}
```



3.5.7 Edge in both directions

```

1 \NewAdigraph{myAdigraph}{
2     s:0,0;
3     t:4,0;
4 }{
5     s,t;
6     t,s;
7 }
8 \myAdigraph{}

```

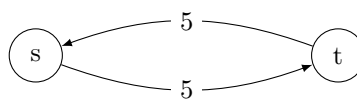


3.5.8 Edge with weights in both directions

```

1 \NewAdigraph{myAdigraph}{
2     s:0,0;
3     t:4,0;
4 }{
5     s,t:5;
6     t,s:5;
7 }
8 \myAdigraph{}

```

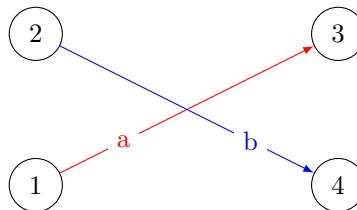


3.5.9 Positioning labels

```

1 \NewAdigraph{myAdigraph}{
2     1:0,0;
3     2:0,2;
4     3:4,2;
5     4:4,0;
6 }{
7     1,3,red:1:a:near start;
8     2,4,blue:1:b:near end;
9 }
10 \myAdigraph{}

```

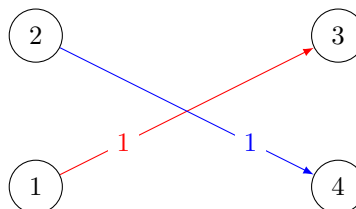


3.5.10 Positioning weights

```

1 \NewAdigraph{myAdigraph}{
2   1:0,0;
3   2:0,2;
4   3:4,2;
5   4:4,0;
6 }{
7   1,3,red:1::near start;
8   2,4,blue:1::near end;
9 }
10 \myAdigraph{}

```

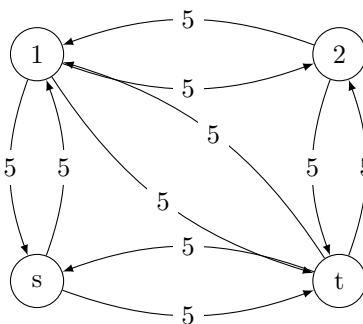


3.5.11 Multiple edges with weights

```

1 \NewAdigraph{myAdigraph}{
2   s:0,0;
3   t:4,0;
4   1:0,3;
5   2:4,3;
6 }{
7   s,t:5;
8   t,s:5;
9   s,1:5;
10  1,s:5;
11  1,2:5;
12  2,1:5;
13  2,t:5;
14  t,2:5;
15  t,1:5;
16  1,t:5;
17 }
18 \myAdigraph{}

```



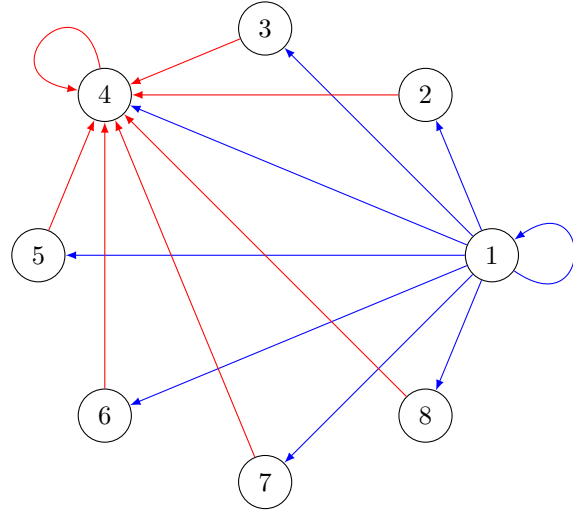
3.6 Kleene star operators

3.6.1 Kleene star on an element

This works only when you don't have a node called j^*_i . When this happens, the behaviour of a tuple j^*_i becomes the normal one.

```

1 \NewAdigraph{myAdigraph}{
2   1:3;
3   2:3;
4   3:3;
5   4:3;
6   5:3;
7   6:3;
8   7:3;
9   8:3;
10 }{
11   1,*,blue;
12   *,4,red;
13 }
14 \myAdigraph{}
```

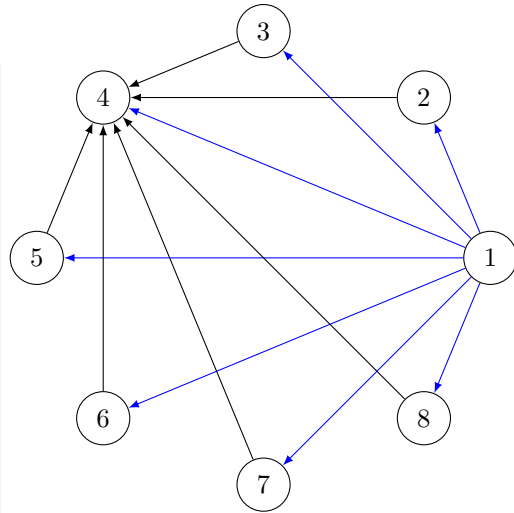


3.6.2 Kleene star minus the element

This works only when you don't have a node called $j+i$. When this happens, the behaviour of a tuple $ja, +i$ becomes the normal one.

```

1 \NewAdigraph{myAdigraph}{
2   1:3;
3   2:3;
4   3:3;
5   4:3;
6   5:3;
7   6:3;
8   7:3;
9   8:3;
10 }{
11   1,+,blue;
12   +,4;
13 }
14 \myAdigraph{}
```



3.6.3 Combining Kleene operations

Sadly, operations such as $i^*, +i$ or $i+, +i$ are not currently supported and not for lack of trying. I'll try implementing them again in the future when I'll have

more time.

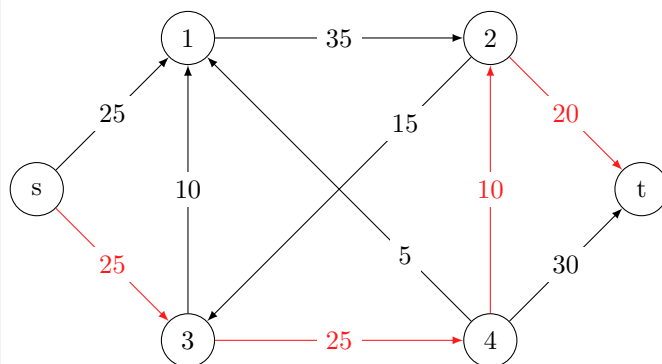
3.7 Paths

A path is specified by the following syntax: *comma separated list of nodes_i*.

```

1  \NewAdigraph{myAdigraph}{
2      s:0,0;
3      1:2,2;
4      3:2,-2;
5      2:6,2;
6      4:6,-2;
7      t:8,0;
8  }{
9      s,1:25;
10     s,3:25;
11     3,4:25;
12     1,2:35;
13     2,t:20;
14     4,t:30;
15     3,1:10;
16     4,2:10;
17     2,3:15::near start;
18     4,1:5::near start;
19 }
20 \myAdigraph{
21     s,3,4,2,t;
22 }

```



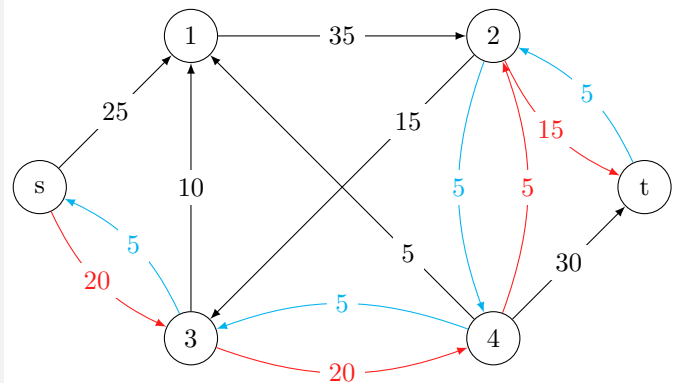
3.7.1 Augmenting paths

An augmenting path is specified by the following syntax: *comma separated list of nodes:units_i*. It is **very important** to note that incremental paths called upon the same object are memorized by default.

```

1  \NewAdigraph{myAdigraph}{
2      s:0,0;
3      1:2,2;
4      3:2,-2;
5      2:6,2;
6      4:6,-2;
7      t:8,0;
8  }{
9      s,1:25;
10     s,3:25;
11     3,4:25;
12     1,2:35;
13     2,t:20;
14     4,t:30;
15     3,1:10;
16     4,2:10;
17     2,3:15::near start;
18     4,1:5::near start;
19 }
20 \myAdigraph{
21     s,3,4,2,t:5;
22 }

```

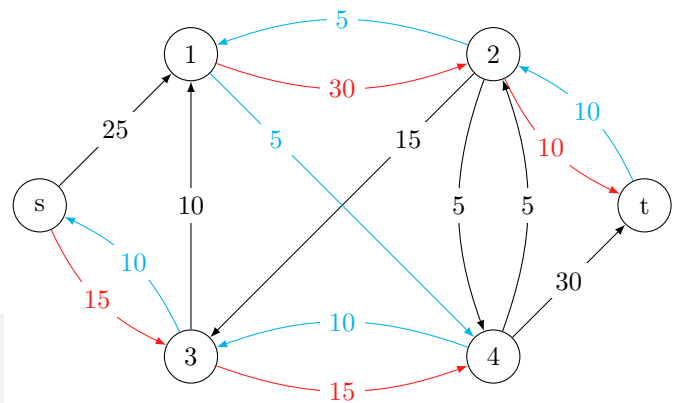


For example, suppose now we'd like to send another 5 units on the graph edited by the previous incremental path, we'll have just to write the following:

```

1  \myAdigraph{
2      s,3,4,1,2,t:5;
3  }

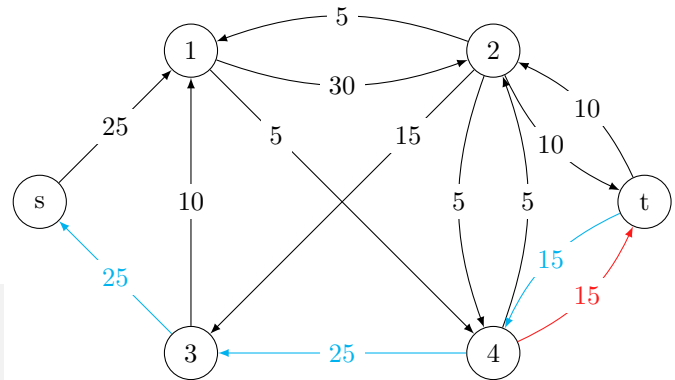
```



```

1 \myAdigraph{
2   s,3,4,t:15;
3 }

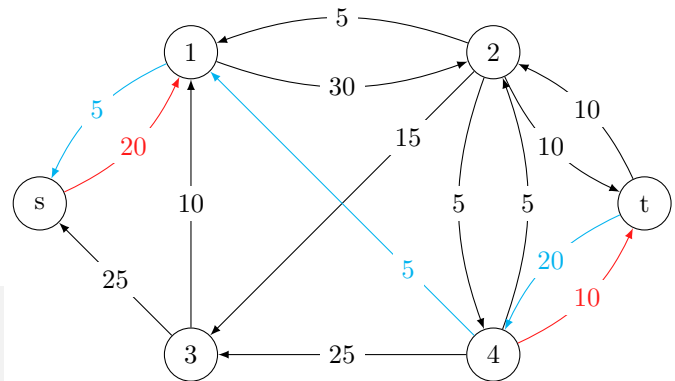
```



```

1 \myAdigraph{
2   s,1,4,t:5;
3 }

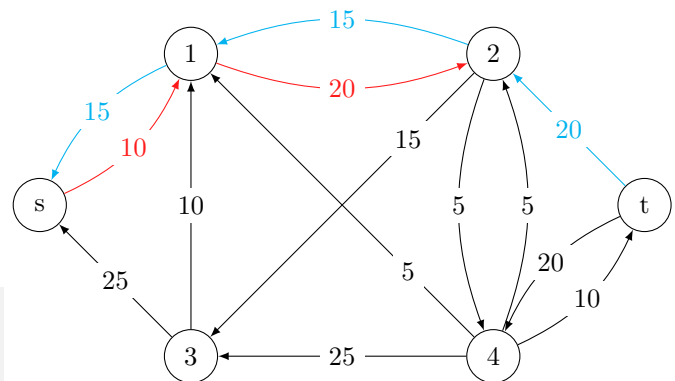
```

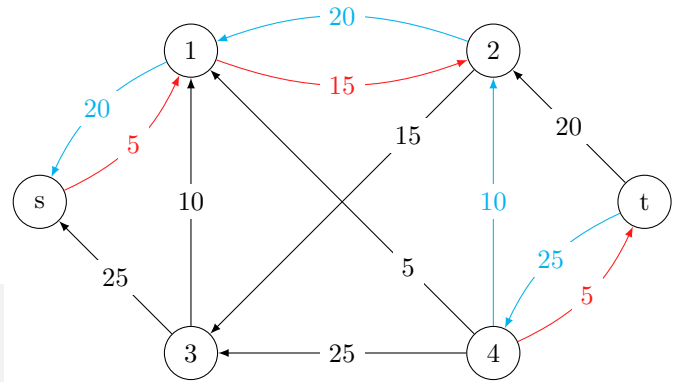


```

1 \myAdigraph{
2   s,1,2,t:10;
3 }

```





```

1 \myAdigraph{
2   s,1,2,4,t:5;
3 }

```

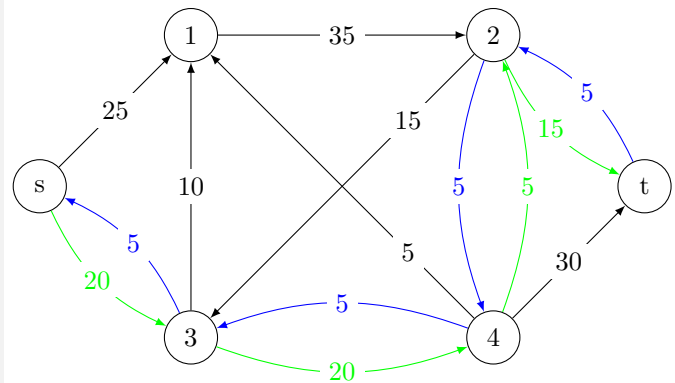
3.7.2 Custom colored augmenting Paths

A path is specified by the following syntax: *j*:*comma separated list of nodes*_{*i*}:*j*units_{*i*}:*j*forward path color, backward path color_{*i*}.

```

1 \NewAdigraph{myAdigraph}{
2   s:0,0;
3   1:2,2;
4   3:2,-2;
5   2:6,2;
6   4:6,-2;
7   t:8,0;
8 }{
9   s,1:25;
10  s,3:25;
11  3,4:25;
12  1,2:35;
13  2,t:20;
14  4,t:30;
15  3,1:10;
16  4,2:10;
17  2,3:15::near start;
18  4,1:5::near start;
19 }
20 \myAdigraph{
21   s,3,4,2,t:5:green,blue;
22 }

```



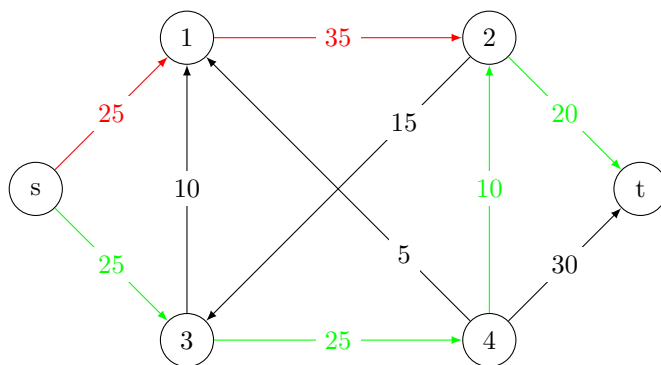
3.7.3 Custom colored Paths

A path is specified by the following syntax: *j*comma separated list of nodes*i*::*j*forward path color, backward path color*i*. **Note the double colons!**

```

1  \NewAdigraph{myAdigraph}{
2      s:0,0;
3      1:2,2;
4      3:2,-2;
5      2:6,2;
6      4:6,-2;
7      t:8,0;
8  }{
9      s,1:25;
10     s,3:25;
11     3,4:25;
12     1,2:35;
13     2,t:20;
14     4,t:30;
15     3,1:10;
16     4,2:10;
17     2,3:15::near start;
18     4,1:5::near start;
19 }
20 \myAdigraph{
21     s,3,4,2,t::green;
22     s,1,2::red;
23 }

```



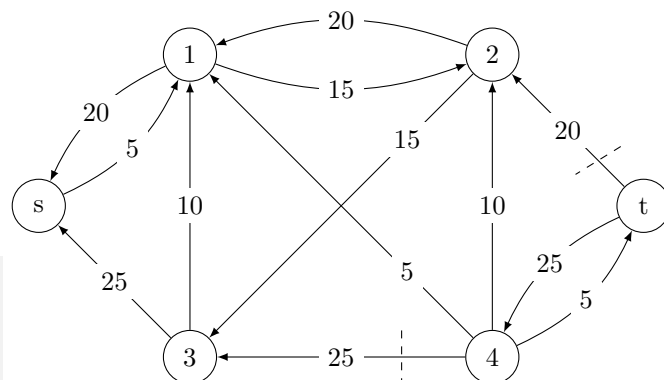
3.8 Cuts

The following is to add cuts to show minimum cuts for example, the syntax is:
*j*first node, second node*i*;

```

1 \myAdigraph{}{
2   3,4;
3   2,t;
4 }

```



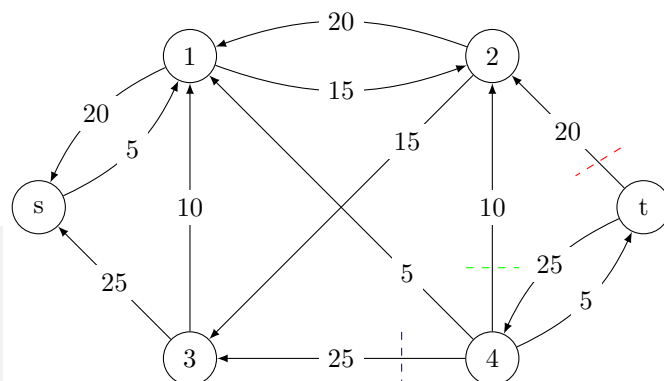
3.8.1 Colored cuts

If you'd like to color the cuts you just have to add the color as follows: *first node, second node, color*; $\dot{c}$. **Note that if you want to only add a cut and not an augmenting path and a cut, you still need to add the empty curly braces {}.**

```

1 \myAdigraph{}{
2   3,4,red;
3   2,t,blue;
4   2,4,green;
5 }

```

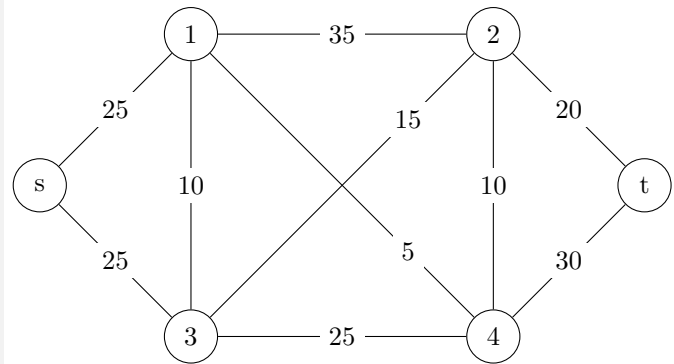


3.9 Non oriented (undirected) edges and custom edge stiles

If you need non oriented edges or in general to ad a custom style to your edges you can proceed as follows:

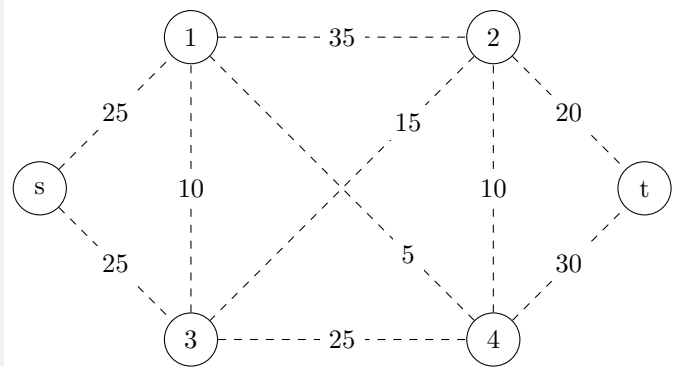
3.9.1 Non oriented (undirected)

```
1  \NewAdigraph{myCustomEdgesAdigraph}{  
2      s:0,0;  
3      1:2,2;  
4      3:2,-2;  
5      2:6,2;  
6      4:6,-2;  
7      t:8,0;  
8  }{  
9      s,1:25;  
10     s,3:25;  
11     3,4:25;  
12     1,2:35;  
13     2,t:20;  
14     4,t:30;  
15     3,1:10;  
16     4,2:10;  
17     2,3:15::near start;  
18     4,1:5::near start;  
19 }[-]  
20 \myCustomEdgesAdigraph{}
```



3.9.2 Dashed

```
1 \NewAdigraph{myCustomEdgesAdigraph}{  
2     s:0,0;  
3     1:2,2;  
4     3:2,-2;  
5     2:6,2;  
6     4:6,-2;  
7     t:8,0;  
8 }{  
9     s,1:25;  
10    s,3:25;  
11    3,4:25;  
12    1,2:35;  
13    2,t:20;  
14    4,t:30;  
15    3,1:10;  
16    4,2:10;  
17    2,3:15::near start;  
18    4,1:5::near start;  
19 }[dashed]  
20 \myCustomEdgesAdigraph{}
```



Chapter 4

PyAdigraph

[Pyadigraph](#) turns your networkx into Adigraph latex package. It requires Adigraph (1.7.0+) to work.

4.1 Installation

The package can be installed by simply running:

```
1 | pip installed pyadigraph
```

4.2 Example

4.2.1 Python code

For example by running the following python code:

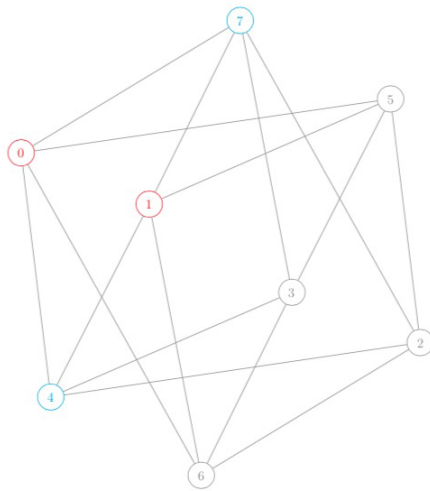
```
1  from pyadigraph import Adigraph
2  import networkx as nx
3
4  A = Adigraph(
5      vertices_color_fallback="gray!90",
6      edges_color_fallback="gray!90",
7      sub_caption="My adigraph number {i} of {n}",
8      sub_label="adigraph_{i}_{n}",
9      row_size=1,
10     caption="A graph generated with python and latex.",
11     label="pyadigraph_example"
12 )
13
14 A.add_graph(
15     nx.bipartite.random_graph(4, 4, 1),
16     vertices_color={
17         0: 'red!90',
18         1: 'red!90',
19         4: 'cyan!90',
20         7: 'cyan!90'
21     })
22
23 A.save("test/result.tex", document=True)
```

4.2.2 Latex result

You automatically obtain the following latex:

```
1 \documentclass{report}
2 \usepackage{adigraph}
3 \usepackage{subcaption}
4
5 \begin{document}
6 \begin{figure}
7     \begin{subfigure}{1.0\textwidth}
8         \NewAdigraph{myAdigraph}{
9             0,red!90,:-0.4386601404141742\textwidth,0.2091077552922947\textwidth;;
10            1,red!90,:-0.15708496776680972\textwidth,0.09630690244229406\textwidth;;
11            2,gray!90,:0.43887677279554366\textwidth,-0.2079924280020609\textwidth;;
12            3,gray!90,:0.15678823839504888\textwidth,-0.09746320565948384\textwidth;;
13            4,cyan!90,:-0.3736460590634439\textwidth,-0.327631363498189\textwidth;;
14            5,gray!90,:0.3735687548614322\textwidth,0.3275275669374224\textwidth;;
15            6,gray!90,:-0.042735184609099336\textwidth,-0.4998552275122768\textwidth;;
16            7,cyan!90,:0.0428925858015027\textwidth,0.5\textwidth;;
17        }{
18            0,4,gray!90,::;
19            0,5,gray!90,::;
20            0,6,gray!90,::;
21            0,7,gray!90,::;
22            1,4,gray!90,::;
23            1,5,gray!90,::;
24            1,6,gray!90,::;
25            1,7,gray!90,::;
26            2,4,gray!90,::;
27            2,5,gray!90,::;
28            2,6,gray!90,::;
29            2,7,gray!90,::;
30            3,4,gray!90,::;
31            3,5,gray!90,::;
32            3,6,gray!90,::;
33            3,7,gray!90,::;
34        }[]
35        \myAdigraph{}
36        \caption{My adigraph number 1 of 1}\label{adigraph_1_1}
37    \end{subfigure}
38    \caption{A graph generated with python and latex.}\label{pyadigraph_example}
39 \end{figure}
40 \end{document}
```

And once you compile that you receive a graph like the following:



(a) My adigraph number 1 of 1

Figure 1: A graph generated with python and latex.

Chapter 5

Utilities

If for some reason you need to disable all the adigraphs in your latex code, for example using an online editor such as Overleaf that allows only a given maximum compile time, you can use the following command:

```
1 | \DisableAdigraphs
```

You can then re-enable adigraphs with:

```
1 | \EnableAdigraphs
```

Chapter 6

Warnings

6.1 Reserved words

I reserve to use for the package the following tokens:

- | | |
|---|--|
| 1. <code>\Adigraph</code> | 18. <code>\AdigraphMemorizeNode</code> |
| 2. <code>\AdigraphBuildEdge</code> | 19. <code>\AdigraphNodeBuilder</code> |
| 3. <code>\AdigraphBuildEdgeWrapper</code> | 20. <code>\AdigraphNodeCounter</code> |
| 4. <code>\AdigraphBuildNode</code> | 21. <code>\AdigraphNodeCounterSecondWrapper</code> |
| 5. <code>\AdigraphBuildNodeWrapper</code> | 22. <code>\AdigraphNodeCounterWrapper</code> |
| 6. <code>\AdigraphBuildPath</code> | 23. <code>\AdigraphNodesCounter</code> |
| 7. <code>\AdigraphCalculateOrientation</code> | 24. <code>\AdigraphPathBuilder</code> |
| 8. <code>\AdigraphCountPaths</code> | 25. <code>\AdigraphProcessAugmentingPaths</code> |
| 9. <code>\AdigraphCutBuilder</code> | 26. <code>\AdigraphProcessAugmentingPathsList</code> |
| 10. <code>\AdigraphDrawEdge</code> | 27. <code>\AdigraphProcessCuts</code> |
| 11. <code>\AdigraphDrawNode</code> | 28. <code>\AdigraphProcessEdges</code> |
| 12. <code>\AdigraphEdgeBuilder</code> | 29. <code>\AdigraphProcessNodes</code> |
| 13. <code>\AdigraphEdgeDrawer</code> | 30. <code>\AdigraphProcessPaths</code> |
| 14. <code>\AdigraphElaboratePath</code> | 31. <code>\AdigraphSimpleSum</code> |
| 15. <code>\AdigraphExecuteCutBuilder</code> | 32. <code>\NewAdigraph</code> |
| 16. <code>\AdigraphGenerateNodeName</code> | 33. <code>\RenewAdigraph</code> |
| 17. <code>\AdigraphMemorizeEdge</code> | 34. <code>\DisableAdigraphs</code> |